CAPSTONE PROJECT • OPTION 2: FEDERATED SMART AGRICULTURE

Hardware-Aware Federated Learning for Edge-Based Crop Disease Detection

Two-member team proposal | Comprehensive simulation with CPU edge profiling

Gao Ge
Student ID: 21337511

Luo Xiaoyu
Student ID: 21341457

Team size: 2; participation is subject to instructor approval as required by the assignment.

1 Problem Statement and Motivation

Leaf diseases threaten crop quality and farm income, while timely diagnosis remains difficult where expert support and network access are limited. Image classifiers trained on pooled data can help, but collecting images from independent farms into a cloud repository creates data-ownership concerns, adds uplink cost, and may reveal crop-health patterns or farm operating conditions. Moreover, a centrally trained model may not adequately reflect variation across crop environments. This project proposes a *federated edge intelligence system* for collaborative tomato-leaf disease recognition, where images remain on participating farm devices and a coordinator aggregates model updates rather than raw images [1, 2].

Our target participants are farmer smartphones, low-power farm gateways, and a cooperative aggregation server. Devices differ in available CPU, memory, network bandwidth and uptime; their image distributions also differ because disease prevalence, lighting and capture conditions vary. The system must therefore tolerate non-identically distributed (non-IID) local data, intermittent participation, and slow clients, while delivering a compact model suitable for offline inference. Federated learning reduces routine raw-data sharing, but *does not by itself guarantee privacy*: model updates can leak information, so the design evaluates clipped/noisy aggregation and states a trusted-server threat model [6].

Research objective. Build and evaluate a reproducible ten-client federated simulation to quantify (i) accuracy under non-IID data, (ii) robustness to heterogeneous resource limits and dropout, and (iii) accuracy–privacy–communication–edge-deployment trade-offs. This is a research prototype, not a clinically or agriculturally certified decision tool.

2 System Architecture Design

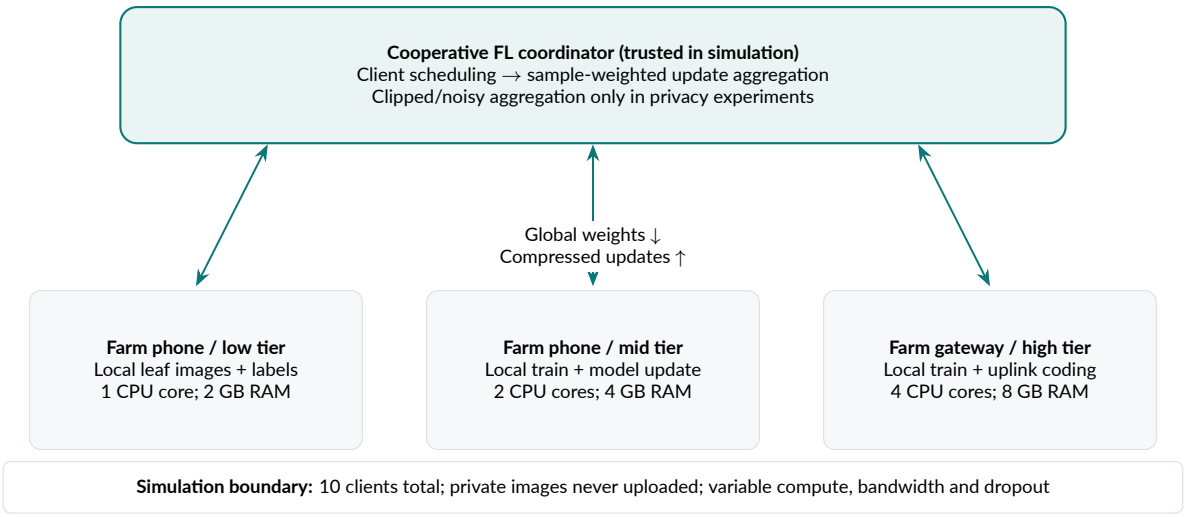


Figure 1: Proposed star-topology federated architecture. Three representative device classes are shown; all ten clients use the same learning task.

2 System Architecture Design (continued)

Topology, participants and data flow

The design uses **centralized cross-silo FL**: ten simulated farms each run a local training process, while one cooperative server coordinates rounds. For every round, the server selects eligible clients, broadcasts the current floating-point model, and specifies a completion deadline. Selected clients train for one local epoch and return model deltas. The baseline uses unmodified updates; privacy experiments apply L_2 clipping, and compression experiments quantize transmitted deltas. The server dequantizes compressed deltas when needed, weights completed updates by local sample counts, and optionally adds noise in a privacy experiment before redistributing the global model. Client-local images and labels are neither uploaded nor logged centrally.

Heterogeneity. The ten clients are assigned to three simulated classes: low-tier phones (1 CPU core, 2 GB RAM), mid-tier phones (2 cores, 4 GB), and farm gateways (4 cores, 8 GB). These are *configured simulation profiles*, not measurements from physical phones. Local batch sizes (16 or 32), available CPU threads, uplink capacity (0.5–10 Mbps), and dropout probabilities (0%, 20%, 40% in stress tests) vary by client. Bandwidth and response times are injected in the simulator; CPU inference is separately benchmarked with actual wall-clock timing. The coordinator maintains per-client availability and completion logs without retaining private examples.

Communication and trust. Flower orchestrates client–server training messages [5]. We account separately for downstream model bytes and upstream delta bytes, adding simulated transfer delay from serialized payload size and client bandwidth. A deployed version would use authenticated TLS links and access-controlled checkpoints. The current threat model assumes the coordinator is trusted to process individual updates; model-update leakage to a malicious coordinator and poisoning-resistant secure aggregation are *outside the guaranteed security scope*. We evaluate a DP-style aggregate-release mechanism as a separate privacy experiment, not as a claim that FL alone is confidential.

System output: a federated global checkpoint, a compressed CPU inference artifact, per-round telemetry (accuracy, participation, bytes, duration), and scripts to reproduce all baselines and ablations.

3 Technical Approach

Learning task, data protocol and aggregation

We will use the publicly available PlantVillage dataset [1] and restrict the main experiment to the **ten tomato-leaf health/disease categories**, resizing images to 160×160 pixels. Before client partitioning, we will freeze train/validation/test splits, group known same-leaf images where identifiers exist, and audit exact or near-duplicate images across splits. We will disclose any leaf identities that cannot be verified. Farm clients are *synthetic partitions*, not actual farm records. For the principal non-IID setting, per-class client proportions are sampled using a Dirichlet distribution ($\alpha = 0.3$); an approximately IID allocation is the control. The assignment seed and client sample counts are recorded. This controlled-image dataset does not establish real-field robustness; if time permits, we will test label-matched, annotated PlantDoc tomato regions as a domain-shift check [8].

The classifier is **MobileNetV3-Small** with a ten-class head [4], trained with cross-entropy and AdamW (batch size 16/32, one local epoch per selected client). **FedAvg** [2] is the default server strategy, with sample-weighted aggregation of completed client parameters:

$$w^{(t+1)} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_k^{(t+1)}.$$

To reduce client drift under non-IID data, we will compare **FedProx** [3], adding $\frac{\mu}{2}\|w - w^{(t)}\|_2^2$ to each client's local objective (initial $\mu = 0.01$, tuned on validation only). Core runs use 10 clients, 5 selected per round, 30 rounds and three random seeds where compute permits. Centralized training and standalone local-only models serve as reference baselines.

3 Technical Approach (continued)

Privacy, model compression and advanced features

Privacy mechanism. In privacy experiments, clients clip model deltas to an L_2 bound C (from training-only pilot runs). The trusted server adds Gaussian noise to the weighted clipped aggregate, with scale reflecting the largest client aggregation weight, before publishing the next global model. We will vary noise strength and record utility loss. A client-level (ε, δ) claim (at $\delta = 10^{-5}$) will be reported *only if* the sampling design, sensitivity bound and multi-round composition are correctly accounted for; otherwise this is a noise-sensitivity experiment, not a formal DP guarantee. The trusted server can still inspect individual updates [6].

Edge compression and deployment. The server keeps FP32 training weights. After each final training run, it exports the selected global model to ONNX, calibrates post-training static INT8 quantization on training-only images, and profiles FP32 versus INT8 using ONNX Runtime on a CPU. We will check operator compatibility and fall back to a supported CPU quantization pathway if needed. Measurements include serialized size, test macro-F1, median/p95 latency, and peak inference memory; the quantized model is *for inference*, not automatically a trainable FL client model [4, 7].

Three planned advanced features.

1. **Heterogeneity-robust optimization (FedProx):** compare against FedAvg as non-IID severity increases; tune the proximal coefficient on validation data.
2. **Deadline-aware, fairness-constrained selection:** prioritize available clients likely to finish the simulated round deadline, while enforcing a minimum participation opportunity for slower farms; compare with uniform random selection.
3. **Quantized client-to-server updates:** apply symmetric 8-bit per-tensor delta quantization with scale metadata and server-side dequantization; compare transmitted bytes, convergence and accuracy against FP32 updates.

Core vs stretch scope. Core: FedAvg, FedProx, client selection, clipped/noisy aggregation ablations, INT8 edge inference and reproducible benchmarks. Stretch: formal privacy-budget accounting (only if valid) and PlantDoc domain-shift testing.

4 Implementation Plan

Environment and milestones

We select **comprehensive simulation**, not a physical multi-farm deployment. Implementation uses Python, PyTorch/torchvision, Flower, NumPy, Hugging Face Datasets, ONNX/ONNX Runtime, and standard timing/resource tools. Development runs on a GPU-capable workstation or available cloud GPU for repeated training; CPU thread limits and artificial delays model client heterogeneity. Pinned dependencies, fixed seeds, partition manifests, configuration files and logged communication counters will support reproduction. Offline CPU inference will be profiled separately from simulated network conditions.

Milestones: **W9** - duplicate audit, frozen split, central/local baselines; **W10** - Flower FedAvg in IID and non-IID settings; **W11** - FedProx and heterogeneous client selection; **W12** - clipping/noisy aggregation and quantized uplink; **W13** - ONNX/INT8 deployment and CPU benchmarks; **W14** - multi-seed ablations, final report and demo. Each stage ends with committed code, test logs and reproducible plots.

4 Implementation Plan (continued)

Team plan, responsibilities and coordination

This is a **two-person project**, subject to instructor approval. Relative to an individual project, the scope includes multiple heterogeneity profiles, an aggregation/selection ablation, explicit privacy sensitivity analysis and an edge-deployment benchmark. Responsibilities are defined below, and all results will be jointly reviewed.

Team member	Primary contributions and accountable artifacts
Gao Ge	Data integrity checks and IID/non-IID partitions; MobileNetV3 baseline; FedAvg/FedProx training pipeline; training-quality and cross-seed analysis.
Luo Xiaoyu	Flower orchestration and client-resource profiles; client scheduling/communication logging; clipped/noisy aggregation experiment; ONNX INT8 and CPU profiling.
Joint work	Weekly integration reviews, interface tests, final experiment matrix, report writing, citations and live demo; changes tracked via shared Git repository and code review.

Risks and mitigations. **(1) Synthetic-domain bias:** PlantVillage contains mostly controlled-background images; clearly limit claims and optionally test overlapping PlantDoc labels. **(2) Non-IID instability:** compare FedProx, vary α, and avoid tuning on the frozen test set. **(3) Unfinished clients and expensive runs:** keep a small ten-client core, cache partitions, start with short pilot rounds, then scale to 30-round runs. **(4) Quantization incompatibility or accuracy loss:** perform operator checks, retain the FP32 artifact, and compare supported static INT8 backends. **(5) Privacy-accounting errors:** independently validate the threat model, sampling scheme and accountant; make no formal guarantee if assumptions fail.

5 Evaluation Methodology

Experiments will use the same frozen test data and three seeds when feasible. The **main comparisons** are (A) local-only model, (B) centralized pooled-data upper reference, (C) FedAvg with random sampling, and (D) FedProx; additional one-factor ablations toggle deadline-aware selection, 8-bit uplink compression, and aggregate noise. Pooled training is a *comparison reference*, not the privacy-preserving system. We will also evaluate client-specific held-out subsets, aligned with the synthetic farm class distributions, instead of reporting only global accuracy.

Dimension	Measurement and comparison
Prediction quality	Global accuracy, macro-F1, per-class recall, worst-client macro-F1; IID vs Dirichlet $\alpha = 0.3$; means $\pm$ SD across seeds.
Convergence/fairness	Macro-F1 versus round, time-to-target, selection counts and completed-client ratio; FedAvg vs FedProx and schedulers.
Systems efficiency	Total uplink/downlink MB, bytes per round, wall-clock round time and simulated 0/20/40% dropout; FP32 vs 8-bit updates.
Edge deployment	FP32 vs INT8 file size, median/p95 CPU latency, peak RAM and post-quantization macro-F1 difference.
Privacy/security	Noise/clipping utility trade-off, valid privacy budget if justified, and explicitly documented server-trust and threat-model limits.

Pre-registered success targets (not observed results). End-to-end FL completion under 20% simulated dropout; non-IID macro-F1 within 5 percentage points of the centralized reference where feasible; at least 50% smaller serialized INT8 inference model with at most 2-point macro-F1 degradation; and at least 50% fewer uplink bytes per client update using 8-bit transmission. We will report missed targets and trade-offs rather than suppressing unsuccessful runs.

Deliverable boundary. The proposal commits to a reproducible *system-level research prototype* and quantitative evaluation; agricultural deployment, trusted-hardware attestation and secure aggregation against an untrusted server are outside this semester's implementation scope.

References

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, art. 1419, 2016. doi:10.3389/fpls.2016.01419.

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. AISTATS*, PMLR, vol. 54, pp. 1273–1282, 2017. https://proceedings.mlr.press/v54/mcmahan17a.html.

[3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated Optimization in Heterogeneous Networks," *Proceedings of Machine Learning and Systems*, vol. 2, 2020. https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html.

[4] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019. https://openaccess.thecvf.com/content_ICCV_2019/html/Howard_Searching_for_MobileNetV3_ICCV_2019_paper.html.

[5] D. J. Beutel *et al.*, "Flower: A Friendly Federated Learning Research Framework," arXiv:2007.14390, 2020. https://arxiv.org/abs/2007.14390.

[6] R. C. Geyer, T. Klein, and M. Nabi, "Differentially Private Federated Learning: A Client Level Perspective," arXiv:1712.07557, 2017. https://arxiv.org/abs/1712.07557.

[7] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018. https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html.

[8] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A Dataset for Visual Plant Disease Detection," in *Proc. ACM CoDS-COMAD*, pp. 249–253, 2020. doi:10.1145/3371158.3371196.